Specification-Driven Development (SDD) + Test-Driven Development (TDD)

A Complete Beginner's Step-by-Step Guide

A complete beginner's step-by-step guide — no prior experience needed

This guide walks you through the entire process from a blank computer to building a feature the correct way — using GitHub's Spec Kit tool (specify-cli). Follow the steps in order. Every command you need to type is shown in a grey box exactly as you should type it.

Why use SDD before TDD?

Traditional development often starts by writing code immediately, which can lead to misunderstood requirements and unnecessary rework. Specification-Driven Development (SDD) helps teams clearly define what needs to be built before implementation begins. Test-Driven Development (TDD) then ensures each requirement is implemented correctly through automated tests.

What You're About To Do

Instead of jumping straight into writing code, this workflow makes you do three quick writing tasks first — a Spec (what to build), a Plan (how to build it), and a Task List (the order to build it in). Only after that do you write any code, and even then you write a small test BEFORE each piece of code, to prove the code actually works.

Think of it like building furniture from a kit: you read the instructions (Spec), check which tools you need (Plan), lay out the steps (Tasks), and then build one piece at a time, checking each piece fits before moving to the next (Tests).

Before You Start — What You Need

Make sure you have these things ready. If you're missing one, ask a teammate or your IT/admin to help you install it.

- A computer with a terminal (Command Prompt / PowerShell on Windows, Terminal on Mac/Linux).
- Python installed (version 3.11 or newer).
- An editor/assistant to work in — this guide covers two: Antigravity (an AI-native editor) or VS Code with GitHub Copilot. Use whichever one your team has set up.

TIP: Antigravity and VS Code both work fine with this workflow — you just need to tell Spec Kit which one you're using so the /speckit slash commands show up in the right place. That's covered in the "Connect Spec Kit To Your Editor" section below, right before Part B.

PART A — One-Time Setup

You only need to do Part A once per computer/project. After that, skip straight to Part B every time you start a new feature.

STEP 1 Install "uv" (the tool that installs everything else)

uv is a fast Python package manager. Open your terminal and type:

```
pip install uv
```

Then check it installed correctly by typing:

```
uv --version
```

You should see a version number printed (e.g. uv 0.x.x). If you see that, you're good — move to Step 2.

STEP 2 Install Spec Kit ("specify-cli")

This is the actual tool that gives you the workflow. Copy and paste this exact command:

```
uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@v0.11.3
```

Windows users only — if the command above doesn't run, use this version instead:

```
py -m uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@v0.11.3
```

STEP 3 Double-check the install worked

```
specify --version
specify check
```

"specify --version" shows the version number. "specify check" scans your computer and tells you if anything else needed is missing. Fix anything it flags before continuing.

STEP 4 Create your project folder

If you're starting a brand-new project, type (replace MY_PROJECT with your project's name):

```
uvx --from git+https://github.com/github/spec-kit.git specify init MY_PROJECT
```

If you already have a project folder open and just want to add Spec Kit to it, type instead:

```
specify init --here
```

If your folder already has files in it and the tool complains, add --force to skip the warning:

```
specify init --here --force
```

During this step, the tool will ask which AI coding assistant you use. Pick the one you have installed. This creates a hidden .specify folder plus some ready-made slash-command files.

Connect Spec Kit To Your Editor (Antigravity or VS Code)

Spec Kit needs to know which editor/assistant you're using, because that decides where it puts the /speckit slash commands. Do this once per project, right after Part A and before you start Part B.

WHAT'S A SLASH COMMAND: A 'slash command' just means typing something starting with / (like /speckit.specify) directly into your assistant's chat box — not into the terminal.

If you're using Antigravity

Run these two commands in your terminal to install and switch on the Antigravity integration:

```
# 1. Install the Antigravity integration
specify integration install agy

# 2. Switch the project's integration to Antigravity
specify integration switch agy
```

Running these removes any Copilot-specific prompt files and registers the /speckit commands under the .agents/skills/ folder instead — which is what makes them show up as slash commands directly inside your Antigravity chat.

If you're using VS Code (with GitHub Copilot)

Run the matching pair of commands instead:

```
# 1. Install the GitHub Copilot integration
specify integration install copilot

# 2. Switch the project's integration to Copilot
specify integration switch copilot
```

This registers the /speckit commands under the .github/prompts/ folder, which is the location GitHub Copilot Chat looks in inside VS Code. These prompt files are only visible to Copilot — not to Antigravity, and vice-versa — so only ever switch to the integration that matches the editor you're actually typing into.

TIP: If you ever switch editors partway through a project (e.g. moving from VS Code to Antigravity), just re-run the "switch" command for the new one. You don't need to reinstall anything you already installed before.

PART B — The 5 Steps For Every New Feature

Important:
Spec Kit assists with generating specifications, plans, and task breakdowns, but developers should always review requirements, validate generated content, and remain responsible for implementation decisions.
Do these steps, in order, every time you want to build a new feature. Steps 1-3 are typed into your assistant's chat window (not the terminal). Steps 4-5 are a manual, repeatable loop you run per task.

STEP 1 Write the Spec — describe WHAT you're building and WHY

In your assistant's chat, type:

```
/speckit.specify Add a login page where users can sign in with email and password
```

Replace the example text with your own feature idea in plain English. Don't mention databases, frameworks, or code — just describe the feature the way you'd explain it to a non-technical friend: what the user sees, what they can do, and what "done" looks like. This creates a file called spec.md.

Before moving on, read spec.md and make sure it correctly captures what you meant. Ask your assistant to fix anything that's wrong.

STEP 2 Write the Plan — decide HOW it will be built

```
/speckit.plan
```

This turns your spec into a technical plan — which language, libraries, and folder structure will be used, and a rough outline of how it'll be tested. This creates plan.md. You don't need to write anything technical yourself — just review the plan and flag anything that looks off.

STEP 3 Break It Into Tasks — a to-do list

```
/speckit.tasks
```

This breaks the plan into a small, ordered checklist (tasks.md) — things like "build the login form," "connect it to the database," and so on. Each item is small enough to finish and test on its own.

STEP 4 Build It Yourself, Task by Task — Manual TDD Loop

IMPORTANT: Do NOT run /speckit.implement. That command lets the AI build the whole feature by itself in one go. Instead, for real control and to actually learn the TDD loop, you drive it manually, one task/spec file at a time, using the steps below.

For each individual task or sub-spec (e.g. "002-product-selection"), repeat this loop:

- a) Ask for the test file first — in your assistant's chat, type something like: "Generate the next TDD file for 002-product-selection. Don't run the tests yourself — tell me when to run them in my terminal."
- b) The assistant creates a test file inside your project's tests folder (e.g. tests/product-selection.spec.ts), based on that task's spec — but writes no feature code yet.
- c) You run the tests yourself in the terminal:

```
npx ng test --watch=false
```

- d) RED — The new tests will fail (this is expected — the feature doesn't exist yet). Copy the failure output from your terminal and paste it back to your assistant.
- e) GREEN — Tell your assistant: "These tests failed — write the minimal code needed to make them pass." It writes the feature code.
- f) Run the same command again to check:

```
npx ng test --watch=false
```

- g) REFACTOR — If tests pass, ask your assistant to clean up the code (better names, remove duplication), then re-run the test command once more to confirm everything is still green.
- h) Repeat steps (a) through (g) for the next task/spec file, until every task in tasks.md has been built and tested this way.

WHY DO IT THIS WAY: This keeps you in control of every test run — nothing executes in your terminal unless you explicitly run it. The assistant only writes files; you decide when to test.

STEP 5 Verify — make sure everything actually works

- Run the full test suite one more time and confirm nothing else broke:

```
npx ng test --watch=false
```

- Confirm the project still builds/compiles with no errors.
- Re-read the original spec.md and check the finished feature really matches what was asked for.

Once all three are confirmed, the feature is done.

If Requirements Change Later

Don't edit the code directly first. Always update in this order: 1) update spec.md, 2) update the tests, 3) then update the code. This keeps your documentation and your code from ever drifting apart.

Quick Reference Checklist (Print This Page)

One-Time Setup

- [ ] pip install uv
- [ ] uv tool install specify-cli --from git+https://github.com/github/spec-kit.git@v0.11.3
- [ ] specify --version and specify check
- [ ] specify init (or specify init --here)

Connect To Your Editor (pick one)

- [ ] Antigravity: specify integration install agy then specify integration switch agy
- [ ] VS Code + Copilot: specify integration install copilot then specify integration switch copilot

Every New Feature

- [ ] 1. /speckit.specify — describe the feature in plain English
- [ ] 2. /speckit.plan — let it generate the technical plan
- [ ] 3. /speckit.tasks — let it break the plan into small tasks
- [ ] 4. For each task: ask for the TDD file → run npx ng test --watch=false → paste failures back → ask for code → re-run → refactor → re-run
- [ ] 5. Verify — run all tests, confirm build works, re-check against the spec

Common Beginner Mistakes to Avoid

- Running /speckit.implement expecting it to build the whole feature safely — for this workflow, build manually task-by-task instead, so you stay in control of each test run.
- Writing or asking for code before a test exists for it — the rule is: no code without a failing test first.
- Mentioning specific technologies (like "use MongoDB") inside the Spec step — that belongs in the Plan step, not the Spec.
- Forgetting to switch the integration (agy vs copilot) when you change editors — the slash commands won't show up in the wrong one.
- Forgetting to re-run specify check after installing — it catches missing pieces before they cause confusing errors later.

Quick Workflow

Idea → Specification → Plan → Tasks → Test (RED) → Code (GREEN) → Refactor → Verify

Glossary — Terms Explained Simply

- **Spec (spec.md):** A plain-English description of what a feature does and why it matters. No technical details.
- **Plan (plan.md):** The technical blueprint — languages, tools, and architecture used to build the spec.
- **Tasks (tasks.md):** A checklist breaking the plan into small, ordered, doable steps.
- **TDD (Test-Driven Development):** Writing a test before writing the code that makes it pass.
- **Red / Green / Refactor:** The three-step loop — write a failing test (Red), write code to pass it (Green), clean up the code (Refactor).
- **Slash command:** A command typed into your assistant's chat box, starting with a forward slash, e.g. /speckit.specify.
- **Integration (agy / copilot):** Tells Spec Kit which editor/assistant should see the /speckit slash commands.

Where These Files End Up

After following this guide, your project folder will contain:

```
.specify/ (Spec Kit's own templates &amp;amp; scripts - don't edit)
.agents/skills/ (slash commands - only if using Antigravity)
.github/prompts/ (slash commands - only if using VS Code + Copilot)
specs/
  001-your-feature-name/
    spec.md (Step 1 output)
    plan.md (Step 2 output)
    tasks.md (Step 3 output)
tests/
  your-feature.spec.ts (created during Part B, Step 4 - run with npx ng test --watch=false)
```

Each new feature you build gets its own numbered specs folder (002-, 003-, ...), so your history of specs stays alongside your code and tests forever.